NEARBY

LOCAL SERVICES MARKETPLACE

A PROJECT REPORT

Submitted by

Susindran D (310624104342)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report "**NEARBY: LOCAL SERVICES MARKETPLACE**" is the Bonafide work of "**Susindran D (310624104342)**" who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. S. Cloudin

Easwari Engineering College

Ramapuram, Chennai - 600089

SUPERVISOR

Ms. Swapna Duggi

Easwari Engineering College

Ramapuram, Chennai - 600089

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce Examination

This is to certify that the Viva Voce Examination of the project work entitled **"NEARBY: LOCAL SERVICES MARKETPLACE"** submitted by Mr. **Susindran D (310624104342)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering was conducted on [VIVA DATE] in the Department of Computer Science and Engineering, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department Dr. S. Cloudin, Department of Computer Science and Engineering, for the continuous encouragement and support.

We are extremely thankful to our Project Guide Ms. Swapna Duggi, Computer Science and Engineering, for valuable guidance, suggestions and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science and Engineering for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	i
2	List of Tables	ii
3	List of Figures	iii
4	List of Abbreviations	iv
5	Chapter 1 INTRODUCTION	1
	1.1 Overview	1
	1.2 Problem Definition	1
	1.3 Objective Of The Project	2
	1.4 Scope Of The Project	2
6	Chapter 2 EXISTING SYSTEM AND PROPOSED SYSTEM	4
	2.1 Existing System	4
	2.2 Limitations Of The Existing System	4
	2.3 Proposed System	4
	2.4 Advantages Of The Proposed System	5
7	Chapter 3 SYSTEM ANALYSIS AND DESIGN	7
	3.1 Hardware And Software Requirements	7
	3.2 System Architecture	8
	3.3 Module Description	10
	3.4 Uml Diagrams And Data Flow Diagram	13
	3.5 Algorithm And Flowchart	16
8	Chapter 4 IMPLEMENTATION	21
	4.1 Implementation Details	21
	4.2 Module Implementation	21
	4.3 Tools And Technologies Used	26

S. No.	Contents	Page No.
9	Chapter 5 RESULTS AND TESTING	29
	5.1 Screenshots And Outputs	29
	5.2 Testing And Validation	32
	5.3 Results And Discussion	35
10	Chapter 6 CONCLUSION AND FUTURE ENHANCEMENT	37
	6.1 Conclusion	37
	6.2 Future Enhancements	37
11	References / Bibliography	39
12	Appendices (Source Code)	40

ABSTRACT

Nearby is a Local Services Marketplace developed to connect customers with service providers through exact six digit Indian pincode matching. Informal service enquiries often separate coverage, price, availability and confirmation across calls and messages. Nearby brings these activities into a centralized web application and links payment status, communication, reviews and disputes to a booking. Its purpose is to provide an organized service journey with clearly defined responsibilities for customers, providers and administrators.

Customers search active services, inspect details and request a scheduled booking. Providers maintain profiles, service areas, listings and date scoped availability, and accept or reject requests. Checkout follows provider acceptance. Administrators manage verification references, marketplace policy, linked payout accounts and disputes. The system uses React, Vite and Tailwind CSS for the interface, Node.js and Express for the API, MongoDB with Mongoose for persistence, and Socket.IO for booking participant messages. JWT and bcrypt support authentication, while password recovery uses an expiring email code and invalidates earlier tokens after a successful reset.

The project contains implementation code for these workflows, but external integration and production readiness require separate verification. The local run on 22 October 2026 recorded 15 passing backend tests and one failing assertion, a successful frontend production build and one frontend lint error. Public interface screenshots were captured from the running application without a connected API. The current development KYC adapter produces a pending reference only; production verification requires a vendor integration. Live payment processing, SMTP delivery and authenticated multi user journeys remain unverified in this report. These evidence limits are retained throughout the results chapter.

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Nearby at a glance	1
1.2	Objectives and how they are met	2
2.1	Comparison of the existing process and Nearby	4
3.1	Functional requirements	7
3.2	Software requirements	8
3.3	Hardware and execution requirements	8
3.4	Layers of the system	9
3.5	Main project folders	9
3.6	Main API route groups	10
3.7	Summary of backend modules	10
3.8	Summary of frontend screens	11
3.9	Background and event driven operations	12
3.10	Database models	12
3.11	Actors and their responsibilities	13
4.1	Libraries frameworks and services	26
4.2	Installation and run commands	27
5.1	Protected screenshot requirements	32
5.2	Test cases and observed outcomes	33
5.3	Test coverage by module	34
5.4	Objectives and outcomes	35
6.1	Roadmap of future work	37
6.2	Representative endpoints	46

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System architecture of Nearby	9
3.2	Use case diagram of Nearby	13
3.3	Database model and relationships	14
3.4	Booking request sequence diagram	15
3.5	Level one data flow diagram	15
3.6	Booking lifecycle states	16
3.7	Authentication and recovery flow	16
3.8	Flowchart of booking hold processing	18
3.9	Flowchart of password recovery	19
5.1	Landing page	29
5.2	Login page	29
5.3	Registration role selection	30
5.4	Customer registration form	30
5.5	Password recovery page	31
5.6	Service discovery page	31
5.7	Pincode validation output	32

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
API	Application Programming Interface
bcrypt	Password hashing function based on the Blowfish cipher
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
HPP	HTTP Parameter Pollution
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
NoSQL	Not only Structured Query Language
ODM	Object Document Mapper
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
YAML	YAML Ain’t Markup Language

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Nearby is a responsive English language marketplace for customers and service providers in India. It supports a service journey from discovery to a recorded outcome: customers find a listing, select a time, request a booking and pay after provider acceptance. Providers publish the services and pincodes they support. Administrators maintain marketplace settings and review exceptional cases.

The system uses exact pincode matching. It does not calculate distance, locate users on a map or perform geocoding. This boundary is reflected in the discovery interface, provider service areas and booking validation. A customer should therefore understand "local" as a matching configured pincode rather than a guaranteed physical radius.

Table 1.1 Nearby at a glance

Item	Description
Project	Nearby Local Services Marketplace
Users	Customer provider and administrator
Frontend	React Vite React Router and Tailwind CSS
Backend	Node.js Express and Socket.IO
Database	MongoDB with Mongoose
Matching rule	Exact six digit Indian pincode
Currency and time	INR integer paise and Asia/Kolkata

1.2 PROBLEM DEFINITION

Service discovery can involve repeated calls to determine coverage, price and availability. Informal arrangements provide little shared history when a booking changes or a disagreement occurs. Providers also need a consistent place to maintain service information and respond to requests. The project addresses this coordination problem by linking operational records to a booking and assigning responsibility to defined roles.

An enquiry is useful only if the customer and provider share the same understanding of the work, service area and schedule. A directory entry alone does not record whether the provider accepted the requested slot. Likewise, a separate payment receipt does not necessarily identify the scope or completion state of the job. Nearby therefore models the booking as a durable coordination record.

A second problem is inconsistent access to operational information. Customers should view their own bookings, providers should manage their own listings and requests, and administrators need controlled access to policy and dispute queues. These responsibilities must be enforced at the API boundary because a browser route or hidden button alone cannot prevent an unauthorized request.

The project also addresses account recovery and exceptional outcomes. A customer who forgets a password needs a bounded recovery path, while cancelled or disputed work needs explicit handling rather than deletion of its history. The implementation uses reset code expiry, attempt limits and token versions, and retains financial and dispute records. Live correctness of the complete journey still requires an integrated test environment.

1.3 OBJECTIVE OF THE PROJECT

The main objective is to provide a structured marketplace for discovering and booking fixed price services. Supporting objectives are to match service coverage accurately, record provider acceptance before checkout, restrict private data to authorized users, and maintain the booking context for messages, reviews and disputes.

Table 1.2 Objectives and how they are met

Objective	**Implementation approach**
Organize discovery	Search active verified provider listings by configured pincode
Coordinate scheduling	Publish date scoped windows and validate reservations
Confirm responsibility	Record provider acceptance before checkout
Protect accounts	Hash passwords and validate JWT identity and role
Support recovery	Use a six digit expiring email code and revoke earlier tokens
Retain service context	Link chat reviews payments and disputes to bookings
Support administration	Maintain policy references resolution queues and audit records

1.4 SCOPE OF THE PROJECT

The implemented scope includes registration, login, profile updates, password recovery, provider onboarding, service listing management, pincode based discovery, availability, booking state transitions, payment integration code, customer reviews,

participant messaging and administration. Prices use INR, with integer paise as the internal representation. Slot dates and times use Asia/Kolkata.

The scope excludes maps, radius search, geocoding, a native mobile application and an implemented production KYC vendor. External account approval, actual payment settlement and delivery of email cannot be established from application source alone. The report separates these dependencies from capabilities visible in the code.

Organization of the report

Chapter 2 compares informal service coordination with the proposed marketplace. Chapter 3 explains requirements, architecture, roles, data relationships and workflows. Chapter 4 describes implementation and configuration. Chapter 5 presents observed interfaces and verification results with their limitations. Chapter 6 gives the conclusion and a prioritized direction for future work. References and an appendix provide supporting documentation and selected technical material.

The report is based on repository inspection, the recorded local checks and actual public browser captures. It does not claim an independently measured improvement in cost, adoption, response time or service quality. External documentation is used to identify the technologies, while project behaviour is explained from implementation evidence and the explicitly reported verification run.

Service coverage and user responsibilities

The discovery contract accepts category, pincode, search text, page and limit parameters. When a pincode is supplied, the route checks a six digit value and requires both the provider profile and the service listing to include that pincode. It filters out inactive listings and providers without verified status. This is a coverage rule maintained by the provider, not a calculation of the physical distance between two addresses. Unsupported query parameters are rejected rather than silently interpreted as a different search method.

The customer is responsible for selecting a suitable service, providing the service address and requesting an available time. The provider remains responsible for maintaining accurate descriptions, coverage and date scoped windows. Administrators configure marketplace policy and review exceptional cases. The application records these decisions, but it does not establish that a provider attended a real appointment or that a customer received satisfactory work merely because a database status changed.

A listing price is a fixed INR amount represented internally in paise. The booking records the agreed amount and duration so that the service context survives later listing changes. The catalogue, schedule and payment configuration therefore work together: a visible service does not by itself establish checkout readiness, and a created request does not establish provider acceptance. These distinctions define the project scope and guide the validation scenarios in Chapter 5.

CHAPTER 2
EXISTING SYSTEM AND PROPOSED SYSTEM

2.1 EXISTING SYSTEM

For comparison, the existing process is an informal workflow in which customers identify providers through personal contacts or directories, then agree on service details by phone or message. This is a problem model used for design; the project has not measured how frequently any particular organization follows it. Availability, payment confirmation and dispute evidence may remain in separate records.

2.2 LIMITATIONS OF THE EXISTING SYSTEM

Customers may need to repeat the same enquiry with several providers because service coverage and free times are not published consistently. Providers receive requests without a common structure, which makes scheduling harder. Without a shared booking identifier, later communication and payment references can be difficult to reconcile. Reviews and complaints may also lack a clear link to completed work.

A fragmented process provides no uniform validation for pincode coverage, selected time or permitted cancellation. It also depends on the participants retaining the same conversation and receipts. The comparison describes a design problem, not a field survey or a measured claim about all service providers. A centralized application can improve record organization only when its data and operating processes are maintained reliably.

2.3 PROPOSED SYSTEM

Nearby uses a central API to validate operations and persist related records. A customer searches for an active listing offered by a verified provider, views the relevant availability and submits a booking request. The provider decides whether to accept the request. Acceptance establishes a temporary hold; checkout then uses configured payment settings. A paid booking proceeds towards completion, with a review period and dispute handling.

The provider maintains profile information, supported pincodes, listings and time windows. The administrator reviews verification references and manages policy settings, linked payout accounts and disputes. The development verification adapter only produces a pending reference. It does not perform identity verification and is disabled as a live verification option in production.

Table 2.1 Comparison of the existing process and Nearby

Aspect	**Informal process**	**Nearby**
Coverage	Individual enquiry	Exact configured pincode

Aspect	Informal process	Nearby
Scheduling	Calls and separate calendars	Date scoped provider windows
Confirmation	Unstructured agreement	Recorded provider decision
Payment record	Separate receipt	Booking linked state
Communication	Personal conversation history	Participant booking conversation
Review	May not identify completed work	Customer completed booking review
Complaint	Scattered records	Dispute and admin resolution queue

2.4 ADVANTAGES OF THE PROPOSED SYSTEM

A shared booking record reduces ambiguity about which service, customer, provider and time a conversation concerns. Exact coverage matching makes the search rule explicit. Integer money values avoid representing paise as floating point fractions. Role and participant checks provide a consistent place to enforce access. Audit records support investigation of administrative and financial changes.

How the proposed system works

A provider creates a profile, declares supported pincodes and publishes a listing with price and duration. Availability windows identify when the provider can accept work.

A customer discovers an eligible service and requests a scheduled booking. The server checks the service area and scheduling inputs before storing the request.

The provider accepts or rejects the request. Accepted work receives a temporary slot hold. Checkout is allowed only when marketplace and payment configuration permit it.

The booking retains the payment and completion context. Eligible participants can communicate, the customer can review completed work, and disputes are reviewed by an administrator.

Feasibility and operating boundaries

The stack is suitable for developing a browser application and a modular API with the tools present in the repository. Operational feasibility depends on reliable MongoDB access, configured email and payment integrations, deployment monitoring and marketplace support processes. Economic feasibility has not been established

through a cost study, and no revenue, adoption or performance projections are claimed in this report.

Operational tradeoffs

Centralized coordination requires maintained coverage, availability and support processes. Database connectivity and configured integrations are prerequisites for transactions. The benefit is a consistent shared record; the software does not itself guarantee service quality or replace marketplace support.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 HARDWARE AND SOFTWARE REQUIREMENTS

Requirements are derived from the current marketplace modules. Functional requirements specify observable responsibilities and access boundaries. They are distinguished from the local tests reported in Chapter 5; a documented requirement is not itself a passing test.

Table 3.1 Functional requirements

ID	Area	Requirement
FR1	Accounts	Register customers and providers and authenticate active accounts
FR2	Access	Restrict role and ownership protected actions
FR3	Recovery	Issue bounded email reset codes and invalidate old tokens
FR4	Providers	Maintain profiles service areas and verification references
FR5	Listings	Create manage and search eligible services
FR6	Availability	Publish date windows and detect conflicting reservations
FR7	Bookings	Record requests decisions checkout and completion
FR8	Payments	Track checkout webhook refund and transfer state
FR9	Communicati on	Restrict messages to booking participants
FR10	Reviews	Allow a customer review for completed work once
FR11	Disputes	Record participant complaints and administrative outcomes
FR12	Administratio n	Manage policy linked accounts and audit records

Non functional requirements

Security requires server side identity and permission checks. Data integrity requires consistent pincode, time and money representations. Usability requires readable states for waiting, errors and unavailable services. Maintainability is supported by domain modules and shared utilities. Reliability, throughput and recovery time are deployment goals which require separate measurement; no performance benchmark is claimed.

Table 3.2 Software requirements

Component	Selection
Operating environment	Windows development workspace with Node.js and npm
Web framework	React 19 Vite 8 React Router 7
Styles and icons	Tailwind CSS 4 project CSS and Lucide icons
API	Node.js Express 5 CommonJS modules
Persistence	MongoDB and Mongoose 9
Realtime and email	Socket.IO 4 and Nodemailer
Verification	Node test runner ESLint and Vite build

Major version families above come from the project manifests. A documentation site may describe a newer release than the installed package. The report therefore uses the local lockfile and command output for implementation observations and the bibliography for general technology reference.

3.2 SYSTEM ARCHITECTURE

Table 3.3 Hardware and execution requirements

Component	Requirement and evidence limit
Computer	Capable of running Node.js browser and development tools
Memory and CPU	No measured minimum or production capacity is established
Storage	Space for dependencies source build artifacts and logs
Network	Connectivity to the selected MongoDB and integration services
Production host	Capacity to be selected through workload and recovery testing

The application separates presentation, API behaviour and persistence. React renders pages and sends requests through the API client. Express validates input and authorization, invokes domain operations and queries MongoDB through Mongoose. Socket.IO connects authorized booking participants. External services are reached through server side configuration and adapters.

This architecture gives a clear boundary between the browser and private credentials. Database connection details, JWT secrets, Razorpay secrets and SMTP passwords belong on the server. Browser environment values beginning with VITE_ are public configuration. A secure architecture also depends on deployment configuration, monitoring and review of the actual runtime behaviour.

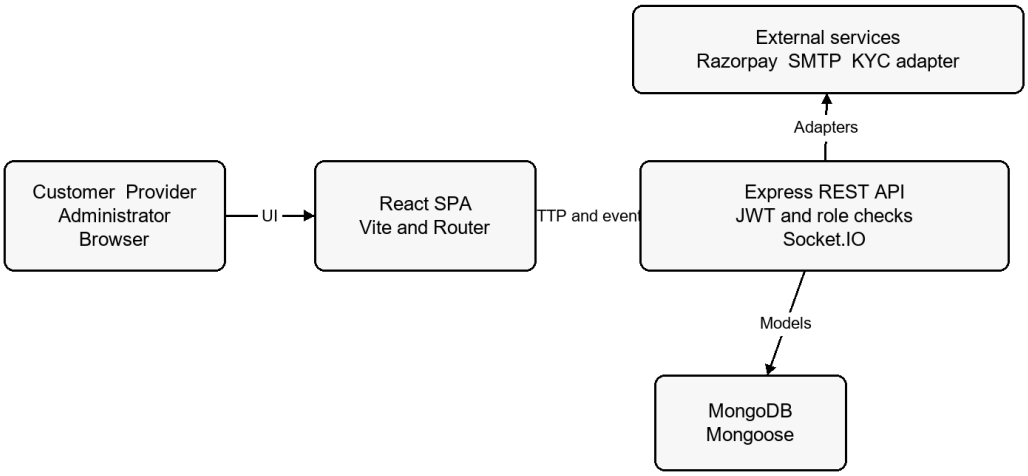


Figure 3.1 System architecture of Nearby

Table 3.4 Layers of the system

Layer	**Responsibility**
Presentation	Public pages protected dashboards and user feedback
Access middleware	Token account role and ownership checks
Domain routes	Service booking payment and dispute operations
Data models	Mongoose schemas references and indexes
Integration	Razorpay SMTP and the KYC adapter contract

The architecture diagram describes implemented boundaries. It does not imply that the external integrations were enabled or exercised during report preparation. The server waits for database connection before listening, and readiness includes a database ping.

Table 3.5 Main project folders

Folder or file	**Purpose**
server/src/app.js	Middleware route groups and readiness response

Folder or file	Purpose
server/src/index.js	Database first startup HTTP realtime and shutdown
server/src/modules	Domain routes services and models
server/src/middleware	Authentication and role checks
server/src/utils	Lifecycle validation and HTTP helpers
server/tests	Focused automated verification
website/src/pages	Public pages authentication and dashboards
website/src/services	Browser API client and session context

Table 3.6 Main API route groups

Group	Responsibility
auth and users	Account access recovery and profile
providers and services	Provider information and marketplace listings
availability and bookings	Time windows reservations and booking decisions
payments	Checkout events verification and refunds
messages reviews disputes	Booking communication feedback and complaints
admin and settings	Operations policy linked accounts and audit

3.3 MODULE DESCRIPTION

Table 3.7 Summary of backend modules

Module	Responsibility
Accounts	Validate registration login password recovery and profiles
Provider profiles	Maintain pincodes and vendor verification references
Services	Store fixed price listings and enforce provider ownership
Availability	Represent windows and date level reservations

Module	Responsibility
Bookings	Coordinate requests acceptance holds and completion
Payments	Manage orders signatures event records refunds and transfers
Messaging	Check participants and persist booking conversation
Reviews and disputes	Store customer feedback and participant exceptions
Administration	Review providers disputes policy accounts and audit data

A modular directory is an implementation organization rather than proof of isolation under all failure conditions. Routes still share a database and utility layer. Cross module actions, such as a refund affecting payout eligibility, need integration tests that span their records and error paths.

Table 3.8 Summary of frontend screens

Screen	Purpose
Landing page	Introduce the product and provide term and pincode search
Service search	Filter categories and display eligible listings
Service details	Present selected service and booking entry
Login and recovery	Authenticate or start a bounded reset flow
Registration	Choose account role and enter required account details
Customer dashboard	Access own bookings and related operations
Provider dashboard	Manage supply profiles listings availability and requests
Administrator dashboard	Access controlled marketplace operations

The same React application hosts these experiences. Protected routing improves navigation, while API checks remain responsible for security. A user with the wrong role is redirected to the appropriate dashboard. The screenshots in Chapter 5 cover actual public states. The report does not replace missing protected session evidence with illustrative screens.

Table 3.9 Background and event driven operations

Trigger	Operation
Lifecycle worker interval	Expire eligible checkout holds and update reservations
Lifecycle worker interval	Complete mature eligible jobs after the review period
Lifecycle worker interval	Reconcile pending refund records
Payment webhook	Verify signature and process an external event
Socket message	Authorize participant persist text and emit message
Reset confirmation	Consume reset state increment token version and close old sockets

The lifecycle worker uses a running guard to avoid overlapping ticks within its process and skips work when the database is disconnected. It uses conditional updates to limit stale state changes. These details do not establish multi instance safety; process coordination, retries and monitoring require a deployment review and integration checks.

Access and private data

The user model excludes password and reset digest fields by default. Provider verification and payout references have restricted exposure. Middleware reloads the active account and checks the token version and role against stored values. This helps reject tokens after reset or account changes, but the complete security posture also depends on secret handling, transport and browser session storage.

Table 3.10 Database models

Model	Stored information
User	Identity role password hash token version and reset state
ProviderProfile	Provider user service areas and verification references
Service	Provider title category fixed price duration and active state
Availability	Provider date scoped windows
AvailabilityDay	Date level reservations and hold state

Model	Stored information
Booking	Customer provider service schedule price and lifecycle
Payment and WebhookEvent	External references events refund and transfer tracking
Message and Review	Booking linked conversation and customer feedback
Dispute and AuditLog	Exceptions resolutions and operational audit
MarketplaceSettings	Public readiness and configured business policy

References are ObjectId values rather than SQL foreign key constraints. Route operations must therefore validate relevant ownership and state. Booking snapshots retain selected commercial details so that later listing edits do not silently change the original request. Index declarations support intended lookups; actual query performance has not been measured in this report.

3.4 UML DIAGRAMS AND DATA FLOW DIAGRAM

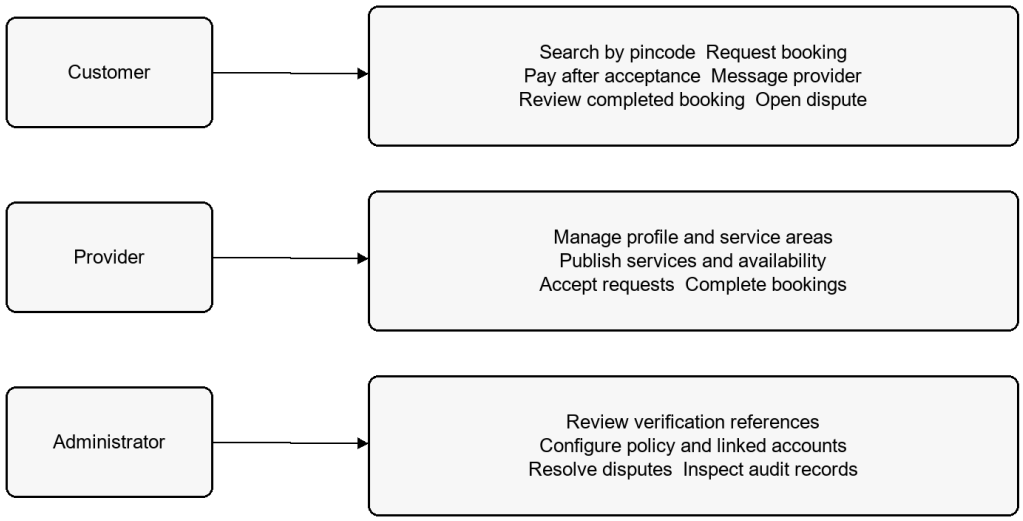


Figure 3.2 Use case diagram of Nearby

Table 3.11 Actors and their responsibilities

Actor	Responsibility
Customer	Discover request pay message review and raise eligible dispute
Provider	Manage profiles coverage listings availability and requests

Actor	Responsibility
Administrator	Review verification references policy linked accounts and disputes
Lifecycle worker	Perform eligible time based booking and refund processing

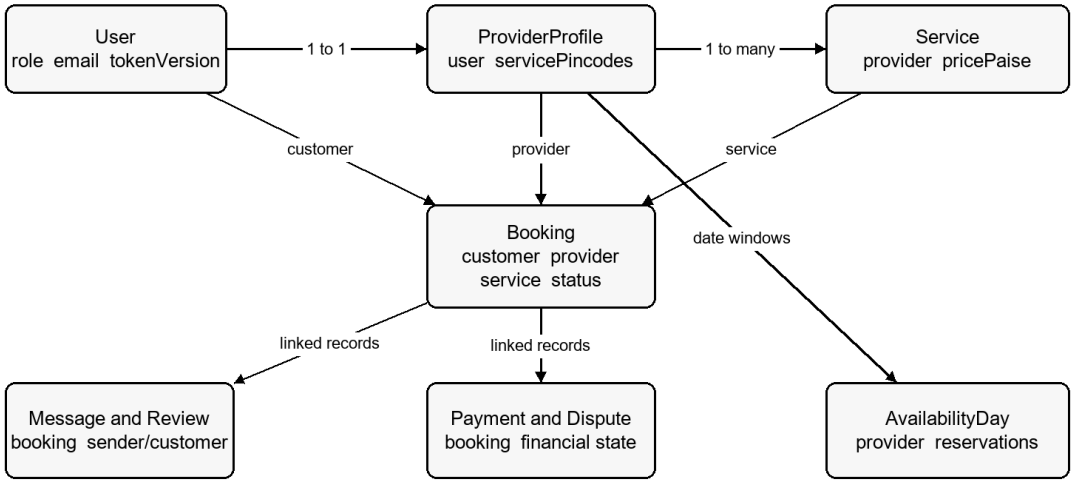


Figure 3.3 Database model and relationships

The diagram shows the principal references around the booking. The provider profile belongs to a user and owns service listings. A booking references its customer, provider and service. Messages, payments, reviews and disputes retain the booking identifier. Availability reservations relate the provider and date to held or confirmed work.

Supporting details are intentionally summarized so the diagram remains readable. It is a document database relationship view and not a complete relational schema. The field level model definitions and route validation are the source of truth for implementation.

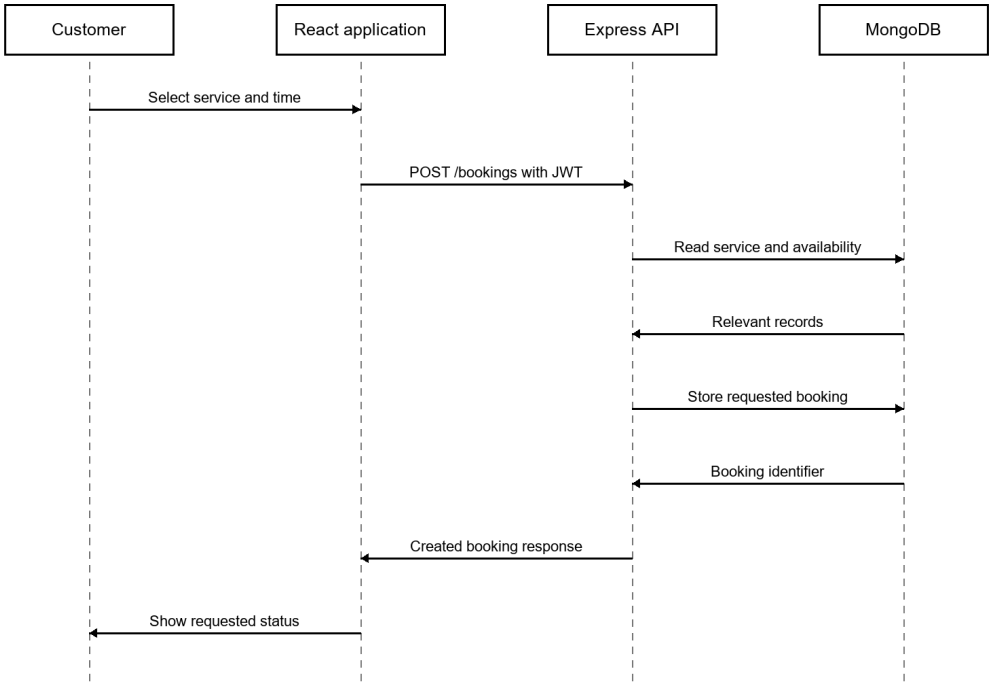


Figure 3.4 Booking request sequence diagram

The sequence describes a request being stored after validation; acceptance and payment are later operations. Each return arrow represents an application response rather than proof of a successful live test.

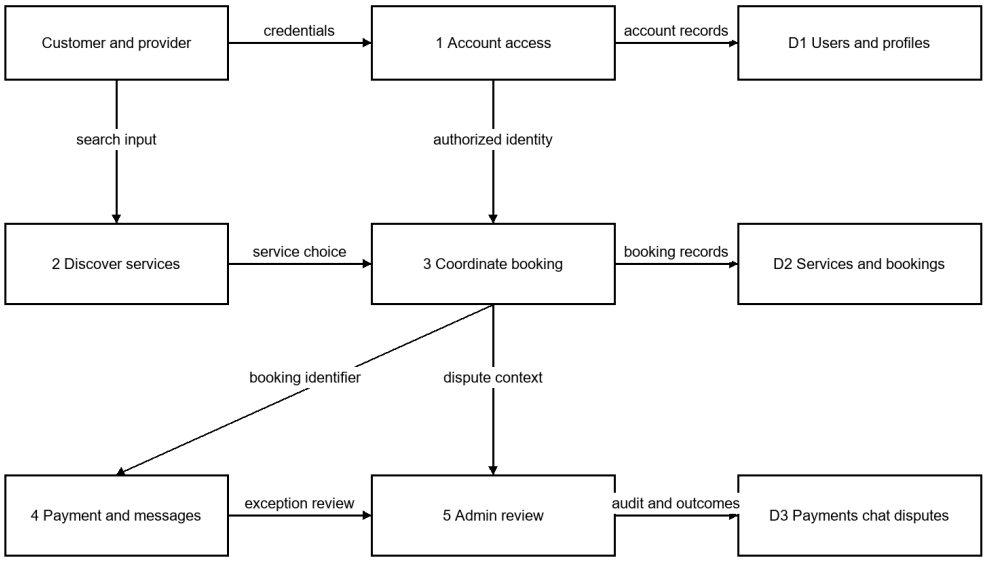


Figure 3.5 Level one data flow diagram

3.5 ALGORITHM AND FLOWCHART

Booking lifecycle and exception outcomes

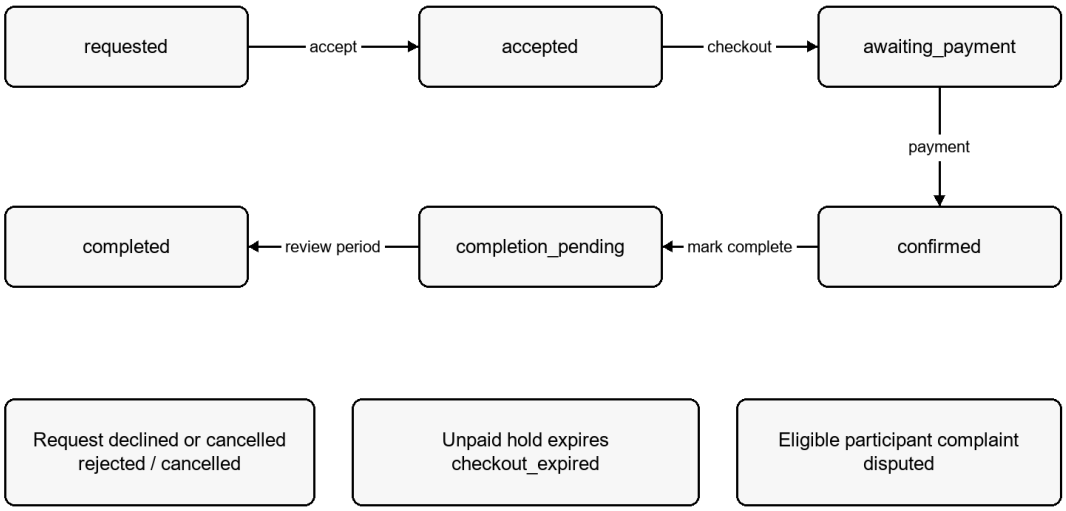


Figure 3.6 Booking lifecycle states

A booking starts as requested. Provider acceptance creates a temporary hold, while rejection closes the request. Checkout begins after acceptance. Successful payment moves the booking to confirmed. A completion action introduces completion_pending and a review deadline before eligible completion. A dispute interrupts normal payout processing. Cancellation and checkout expiry are additional terminal or exception outcomes.

This figure summarizes major paths; individual routes enforce their own state and participant checks. It is not an assertion that every displayed state can transition directly to every exception state. Unpaid holds are released after expiry. Refund state is tracked with the payment record, and provider transfers remain held when refund reconciliation or an open dispute requires it.

Sign in and role based authorization algorithm

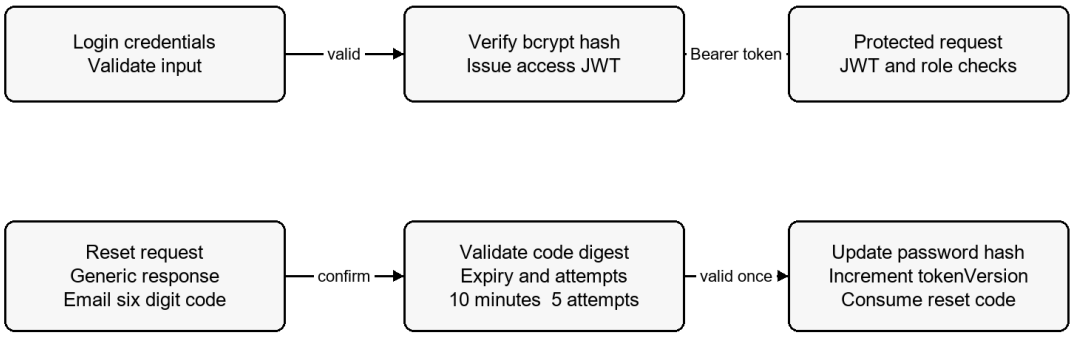


Figure 3.7 Authentication and recovery flow

Login validates credentials against a bcrypt hash and issues a signed JWT. Protected API requests verify the token and relevant role. Token versions provide a way to invalidate earlier access tokens after password recovery. A reset request returns the same account safe message whether or not an active account exists.

The reset code is six decimal digits generated with cryptographic randomness. The application stores a keyed digest with an expiry ten minutes in the future and a maximum of five failed attempts. A valid confirmation hashes the new password, consumes the reset fields and increments tokenVersion. Existing messaging sessions are also disconnected where possible. Actual receipt of email depends on SMTP configuration.

Account state and role changes

Authentication is checked against current account data rather than only the role embedded in an old token. Middleware verifies that the user still exists, is active and has the claimed role, then compares token versions. Password recovery increments the version so that earlier access tokens fail subsequent checks. Domain operations add their own ownership or participant restrictions. A valid customer token therefore does not grant access to another customer’s booking, and a valid provider token does not grant editing rights over another provider’s listing.

Availability hold and checkout algorithm

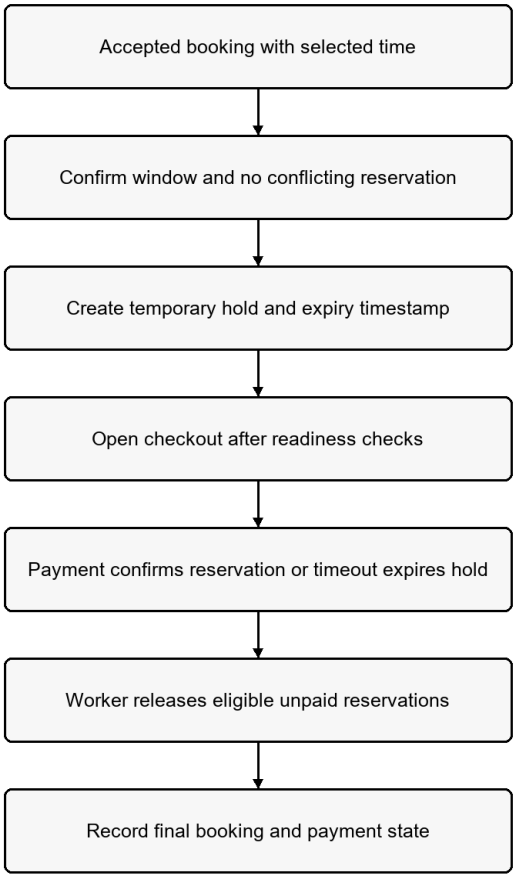


Figure 3.8 Flowchart of booking hold processing

Window validation and reservation checks must occur before checkout proceeds. A hold is temporary, while a confirmed payment retains the reservation. The worker expires only eligible unpaid holds. Failed external operations and reconciliation need separate test scenarios rather than an assumed successful transition.

Password reset and session invalidation algorithm

A request accepts the email address and returns an account safe response. For an active account and usable secret, the server generates a six digit code, stores a keyed digest and records an expiry ten minutes ahead. The code is sent through SMTP; a failed or unavailable delivery clears the matching stored reset state.

Confirmation reads the private reset fields. Expired codes and codes beyond the attempt limit cannot change the password. An incorrect code increments the counter conditionally, and reaching five attempts clears the code. A valid code proceeds to a password hash and a conditional update that consumes the digest, changes the password and increments tokenVersion.

The conditional update binds the operation to the same active code and its remaining time and attempt budget. A later request cannot simply reuse a consumed

code. Authentication compares the token version against the current user, so earlier JWTs fail after successful recovery. Existing booking message sockets are disconnected where possible.

Money and time consistency

Listing prices are accepted in rupees and converted to integer pricePaise values. Bookings retain the agreed price and duration rather than relying solely on a later editable listing. Provider dates and slot times are interpreted in Asia/Kolkata. The model supports explicit financial and scheduling rules without introducing geolocation based matching.

Password recovery flowchart

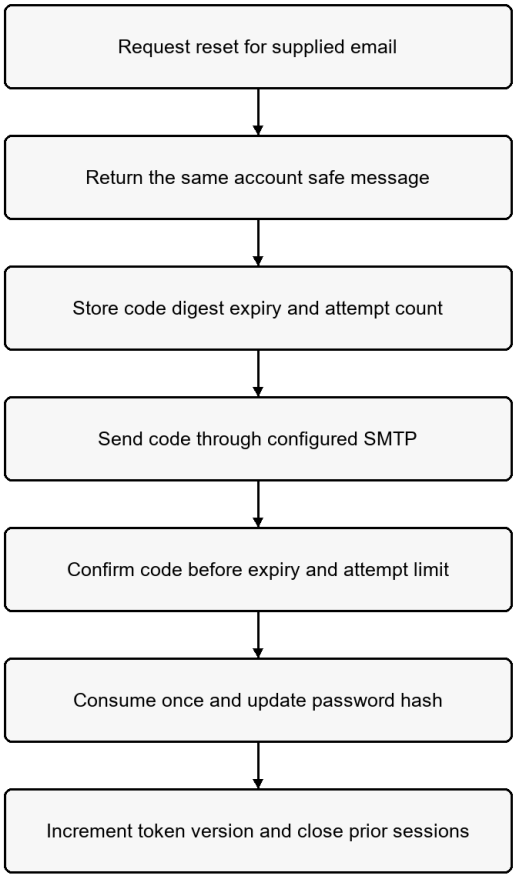


Figure 3.9 Flowchart of password recovery

This is the intended implementation path. The focused automated reset tests passed, but receipt of an actual email and completion in a browser were not verified. The flowchart must therefore be read with the evidence boundary in Chapter 5.

Failure behaviour

When verification, payment configuration or email delivery is unavailable, the associated workflow must be reported as unavailable or incomplete. It must not be treated as successful because the interface rendered. Account recovery, booking holds and refunds each retain their own state and failure conditions, which require separate tests before live operation.

CHAPTER 4
IMPLEMENTATION

4.1 IMPLEMENTATION DETAILS

The server entry point loads configuration, connects to MongoDB, creates the HTTP and Socket.IO services and starts lifecycle processing. The default API port is 4000. The frontend runs through Vite on port 5173 and reads a browser safe API base URL. The configured local web origin should use localhost consistently with the CORS allowlist.

Environment examples document required settings without exposing real values. MONGO_URI supplies database access and JWT_SECRET signs authentication tokens. CLIENT_ORIGINS defines permitted browser origins. Razorpay and SMTP settings stay on the server. The VITE_API_URL variable is public configuration and must not contain private credentials.

Database connection and startup

Code snippet 4.1 Database connection configuration

```
const mongoose = require("mongoose");
const dns = require("node:dns");
const net = require("node:net");

function configureDnsResolvers() {
  const configured = process.env.MONGODB_DNS_SERVERS;
  if (!configured) return;

  const servers = configured.split(",").map((server) => server.trim());
  if (!servers.length || servers.some((server) => !net.isIP(server))) {
      throw new Error("MONGODB_DNS_SERVERS must contain comma-separated IP
addresses.");
  }

  dns.setServers(servers);
}

async function connectdb(dbUrl) {
  if (!dbUrl) {
    throw new Error("MONGO_URI is required to start the API.");
  }

  configureDnsResolvers();
  await mongoose.connect(dbUrl, {
```

4.2 MODULE IMPLEMENTATION

Authentication and account creation

Registration validates name, email, phone, country code, password and role. Phone input contains seven to fifteen digits and the country code uses a leading plus sign. Passwords must contain at least eight characters and fit the bcrypt compatible limit of

72 UTF 8 bytes. Public registration accepts customer and provider roles. Password hashes are created with bcrypt and returned account data excludes the password hash.

The observed backend test failure is explained by validation order: its attempted administrator registration omits the now required phone value, so the endpoint returns HTTP 400 before reaching the forbidden role check that the test expects to return HTTP 403. The request is rejected, but this test does not currently isolate the intended role restriction. No application or test code was changed while preparing this report.

Code snippet 4.2 Account creation and password hashing

```
try {
   const normalizedEmail = normalizeEmail(email);
   const existing = await User.exists({ email: normalizedEmail });
   if (existing) return sendError(res, 409, "EMAIL_IN_USE", "An account already exists
for this email address.");


   const passwordHash = await bcrypt.hash(password, 12);
      const user = new User({ name: name.trim(), email: normalizedEmail, phone,
countryCode, passwordHash, role, isActive: true });
   const token = createAccessToken(user);
   await user.save();


   return sendSuccess(res, {
     token,
     user: { id: String(user._id), name: user.name, email: user.email, phone: user.phone,
countryCode: user.countryCode, role: user.role },
   }, 201);
 } catch (error) {
```

Role based authorization middleware

requireAuth parses a Bearer token, requires a configured secret and verifies HS256 claims. It then checks that the referenced active user still has the claimed role and token version. requireRole applies the route role restriction after identity has been established. Ownership and participant checks are additional domain rules, not replacements for token validation.

Code snippets 4.3 and 4.4 Account validation and role restriction

```
|| !["customer", "provider", "admin"].includes(claims.role)) {
      return sendError(res, 401, "INVALID_TOKEN", "The access token is invalid or
expired.");
  }


  try {
   const user = await User.findById(claims.sub).select("name email phone countryCode
role isActive tokenVersion").lean();
   const tokenVersion = Number.isSafeInteger(user?.tokenVersion) ? user.tokenVersion
: 0;
   if (!user || !user.isActive || user.role !== claims.role) {
          return sendError(res, 401, "ACCOUNT_UNAVAILABLE", "This account is
unavailable.");
   }
   if ((claims.ver ?? 0) !== tokenVersion) {
       return sendError(res, 401, "INVALID_TOKEN", "The access token is invalid or
expired.");
   }
```

```
    req.user = { id: String(user._id), role: user.role, email: user.email };
    return next();
  } catch (error) {
    console.error("Authentication lookup failed:", error.message);
       return sendError(res, 500, "AUTH_LOOKUP_FAILED", "Unable to validate the
account.");
  }
// Intervening source lines omitted
function requireRole(roleOrRoles) {
  const roles = Array.isArray(roleOrRoles) ? roleOrRoles : [roleOrRoles];
  return (req, res, next) => {
       if (!req.user) return sendError(res, 401, "UNAUTHENTICATED", "Sign in to
continue.");
    if (!roles.includes(req.user.role)) return sendError(res, 403, "FORBIDDEN", "You do
not have access to this action.");
    return next();
  };
}

module.exports = { requireAuth, requireRole };
```

Provider profiles and service management

Provider profiles hold service coverage and verification references. Listings expose a category, service details, fixed price and duration. Public discovery limits results to active listings from verified providers and applies exact pincode matching. Provider only listing operations enforce ownership. Availability publishes date scoped time windows and checks conflicts against reservations and bookings.

The KYC adapter has an explicit disabled mode. In development it can return a pending reference without receiving documents. In production the current adapter reports verification unavailable. Selecting a vendor and implementing its production integration is required before treating the marketplace as a verified provider onboarding service.

Code snippet 4.5 KYC adapter selection

```
if (!configuredProvider || configuredProvider === "development_stub" ||
configuredProvider === "mock") {
    return developmentAdapter;
  }
  return disabledAdapter;
}

module.exports = { disabledAdapter, developmentAdapter, getKycAdapter };
```

Availability and reservation handling

Availability stores provider windows with explicit dates and time ranges. The helper converts local values using the India offset and checks that a booking fits in one local calendar day. Date level reservations represent held and confirmed work. Overlap checks compare interval starts and ends, while expired held reservations do not permanently block the provider.

Code snippet 4.6 Reservation overlap predicate

```
const now = new Date();
```

```
  const noOtherOverlap = {
    provider: providerId,
    dateKey: local.dateKey,
    reservations: {
      $not: {
        $elemMatch: {
          booking: { $ne: bookingId },
          startsAt: { $lt: endsAt },
          endsAt: { $gt: startsAt },
          $or: [
            { state: { $in: ["confirmed", "completed"] } },
            { state: "held", expiresAt: { $gt: now } },
          ],
        },
      },
    },
  };

  const existing = await AvailabilityDay.findOne({
    provider: providerId,
    dateKey: local.dateKey,
    "reservations.booking": bookingId,
  }).lean();
   const ownReservation = existing?.reservations?.find((item) => String(item.booking)
=== String(bookingId));
```

The update condition prevents another active overlapping reservation in the same provider day record. Concurrent requests, expiry races and retry handling still need live database tests before drawing conclusions about production scheduling reliability.

Booking requests and state transitions

Booking creation validates the service, service area, selected time and availability. Provider acceptance, rejection, checkout, completion and cancellation are separate operations. The booking stores its price snapshot and lifecycle timestamps. Checkout readiness depends on marketplace policy, payment configuration and a provider linked account rather than a frontend button alone.

Payment integration code handles Razorpay orders, signature checks, webhook event deduplication, refund tracking and held Route transfers. For a partial cancellation refund, commission applies to the amount retained; the provider receives the remaining cancellation fee. A full refund leaves no provider transfer. External processing and settlement were not exercised during this report run.

Code snippet 4.7 Booking status and lifecycle timestamps

```
},
    pricePaise: { type: Number, required: true, min: 1 },
    durationMinutes: { type: Number, required: true, min: 1 },
    slotHoldExpiresAt: { type: Date, default: null, index: true },
    completionMarkedAt: { type: Date, default: null },
    completionMarkedBy: { type: mongoose.Schema.Types.ObjectId, ref: "User", default:
null },
    reviewDeadlineAt: { type: Date, default: null, index: true },
    completedAt: { type: Date, default: null },
    cancelledAt: { type: Date, default: null },
    cancellationReason: { type: String, trim: true, maxlength: 1000, default: "" },
  },
  { timestamps: true }
```

```
);

bookingSchema.index({ customer: 1, createdAt: -1 });
bookingSchema.index({ provider: 1, scheduledAt: 1, status: 1 });
bookingSchema.virtual("review", {
  ref: "Review",
```

Payment processing and financial reconciliation

The payment routes accept webhook bytes before ordinary JSON parsing so that signature verification can use the original payload. Webhook event records help handle duplicates, while payment records track order creation, refunds and held transfers. Configuration determines whether checkout is ready; no sandbox payment was performed while preparing this report.

Code snippet 4.8 Webhook entry point

```
router.post(
  "/webhook",
  asyncHandler(async (req, res) => {
    const rawBody = Buffer.isBuffer(req.body) ? req.body : req.rawBody;
     ensure(Buffer.isBuffer(rawBody), 400, "Webhook requires the raw request body", "RAW_WEBHOOK_BODY_REQUIRED");
    const secret = process.env.RAZORPAY_WEBHOOK_SECRET;
          ensure(secret, 503, "Razorpay webhook secret is not configured", "PAYMENTS_NOT_CONFIGURED");
    const received = req.get("x-razorpay-signature");
       ensure(typeof received === "string" && /^[a-f\d]{64}$/i.test(received), 401, "Webhook signature is missing or invalid", "INVALID_WEBHOOK_SIGNATURE");
                              const expected = crypto.createHmac("sha256", secret).update(rawBody).digest("hex");
    const expectedBuffer = Buffer.from(expected, "hex");
    const receivedBuffer = Buffer.from(received, "hex");
                    ensure(expectedBuffer.length === receivedBuffer.length && crypto.timingSafeEqual(expectedBuffer, receivedBuffer), 401, "Webhook signature is invalid", "INVALID_WEBHOOK_SIGNATURE");


    let payload;
    try {
      payload = JSON.parse(rawBody.toString("utf8"));
    } catch (_error) {
              throw new LifecycleError(400, "Webhook body is not valid JSON", "INVALID_WEBHOOK_BODY");
    }
    const eventName = payload?.event;
        ensure(typeof eventName === "string", 400, "Webhook event is missing", "INVALID_WEBHOOK_BODY");
                          const eventId = req.get("x-razorpay-event-id") || crypto.createHash("sha256").update(rawBody).digest("hex");
    const result = await processWebhookEvent(eventId, eventName, payload);
           res.status(200).json({ success: true, data: { received: true, duplicate: result.duplicate } });
```

Partial cancellation calculations apply commission to the retained balance. The focused arithmetic tests pass for the recorded scenarios. They do not establish that the external gateway accepted a refund or that a provider received a transfer. Those outcomes require separately captured sandbox evidence.

Booking messages reviews and disputes

Messaging opens after provider acceptance for the supported booking states. REST and Socket.IO operations verify booking participation. Message text is trimmed and limited to 4000 characters. Realtime connection authentication uses auth.token; booking:join selects the room, message:send submits a message and message:new delivers new records.

Review creation is customer only and requires the customer's completed booking, a rating from one to five and a unique booking review. Disputes are participant requests reviewed through administrative operations. An open dispute pauses payout. Administrative and financial actions record audit information to support later review; this does not substitute for an operational support process.

Code snippet 4.9 Participant message validation

```
async function createMessage(bookingId, senderId, rawText) {
  ensure(isObjectId(bookingId), 400, "Invalid booking ID", "INVALID_BOOKING_ID");
  const text = typeof rawText === "string" ? rawText.trim() : "";
  ensure(text.length > 0 && text.length <= 4000, 400, "Message must be between 1 and
4000 characters", "INVALID_MESSAGE");
  const booking = await Booking.findById(bookingId);
  ensure(booking, 404, "Booking not found", "BOOKING_NOT_FOUND");
  ensure(MESSAGE_STATUSES.includes(booking.status), 409, "Messaging opens after a
provider accepts the booking", "MESSAGING_NOT_AVAILABLE");
  ensure(await canParticipate(booking, { id: senderId }), 403, "Only booking participants
can access this conversation", "FORBIDDEN");
  const message = await Message.create({ booking: booking._id, sender: senderId, text
});
   const populated = await Message.findById(message._id).populate("sender", "name
role").lean();
  return { booking, message: populated };
}

async function notifyRecipient(booking, senderId, text) {
  let recipientId = booking.customer;
  if (String(booking.customer) === String(senderId)) {
```

4.3 TOOLS AND TECHNOLOGIES USED

Table 4.1 Libraries frameworks and services

Technology	Use
React and React Router	Components route selection and protected views
Vite and Tailwind CSS	Development bundle tooling and responsive styling
Express and Mongoose	Domain endpoints schemas and persistence
JWT bcryptjs and Zod	Token validation password hashes and request checks
Socket.IO and Nodemailer	Booking events and configured SMTP delivery

Technology	Use
Razorpay Checkout and Route	Checkout linked accounts and held transfers

Frontend data access

React Router defines public landing, search, service detail, login, registration and password recovery pages. Protected routes provide customer, provider and administrator dashboards. Session context supplies the current user and readiness state. ProtectedPage redirects unauthenticated users to login and sends a user with a mismatched role to the appropriate dashboard.

The visual system uses a white canvas, yellow navigation and highlighted surfaces, blue actions and dark text. Search accepts a service term and a six digit pincode. Forms and API components expose loading and error states. The public screenshots in Chapter 5 show the running interface; protected workflows were inspected in source but were not exercised with authenticated accounts in this report run.

Table 4.2 Installation and run commands

Command and folder	Purpose
npm install in server	Install API dependencies
npm run dev in server	Run the database connected development API
npm test in server	Run existing focused backend tests
npm install in website	Install browser application dependencies
npm run dev in website	Start Vite development server
npm run lint in website	Check frontend code using ESLint
npm run build in website	Generate the production browser bundle

Deployment and integration configuration

The repository describes deploying the web application with website as the Vercel root and the server as a Node service on Render. These are deployment instructions, not verified active deployments. Production requires HTTPS, secrets management, database network access, email delivery, payment approval, operational monitoring and tested backup recovery. Local build success establishes a generated web bundle only.

Keep localhost consistent with allowed origins during local testing. Set the public web API base URL to the deployed /api/v1 URL only after configuring the backend origin and HTTPS. Deploying a frontend alone does not establish database readiness, payment approval, SMTP delivery or successful administration of disputes.

API errors and integration readiness

The browser API client reads its token from session storage and attaches it as a Bearer header. It distinguishes a failed network request from an HTTP response and attempts to parse JSON before returning data. Network failures produce a message asking the user to check API availability. The current client mainly expects a structured error object, while some lifecycle routes return an error string with a separate code. Broader integration testing should check whether useful failure messages reach the interface in both cases.

Marketplace settings store commission, checkout timeout, completion review and cancellation policy values, together with the Route enabled flag. Several values begin unset, and Route is disabled by default. These fields are operational configuration, not evidence that a particular commission or refund percentage has been approved. An operator must configure the intended policy and verify the external payment account before enabling checkout. The report therefore separates configured readiness from the existence of payment handling code.

CHAPTER 5
RESULTS AND TESTING

5.1 SCREENSHOTS AND OUTPUTS

These are actual public browser captures from the running local application. They show visible states rather than successful authenticated transactions. The database connected API was unavailable during capture; no account or payment was created for the report.

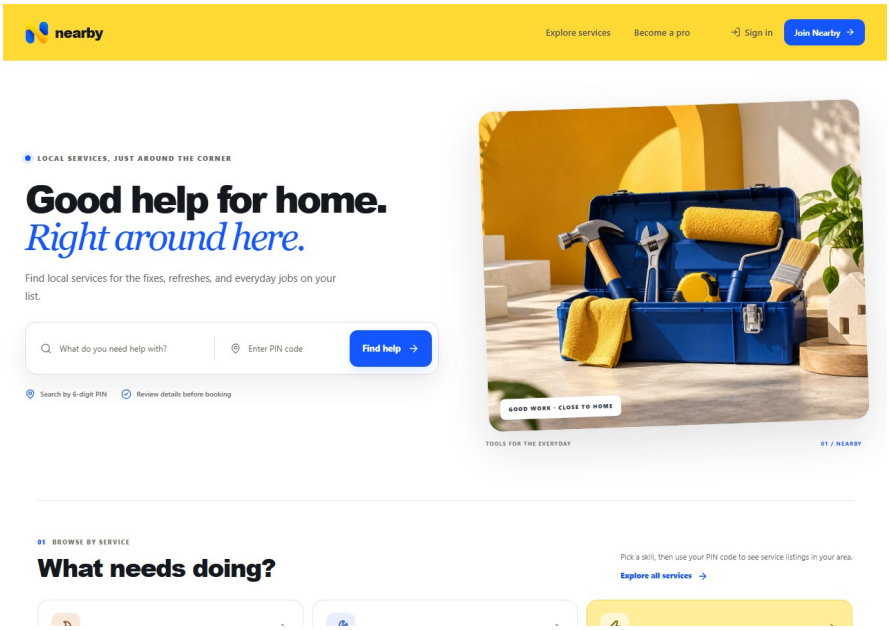


Figure 5.1 Landing page

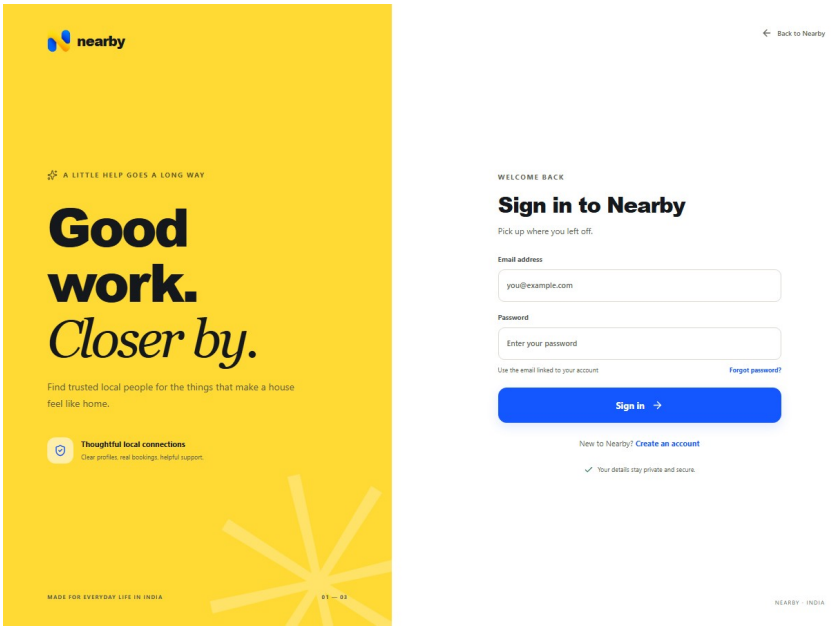


Figure 5.2 Login page

The login page links to registration and recovery. This capture contains no credentials or session token; successful authentication still requires an active account and a connected API.

Registration role selection and account form

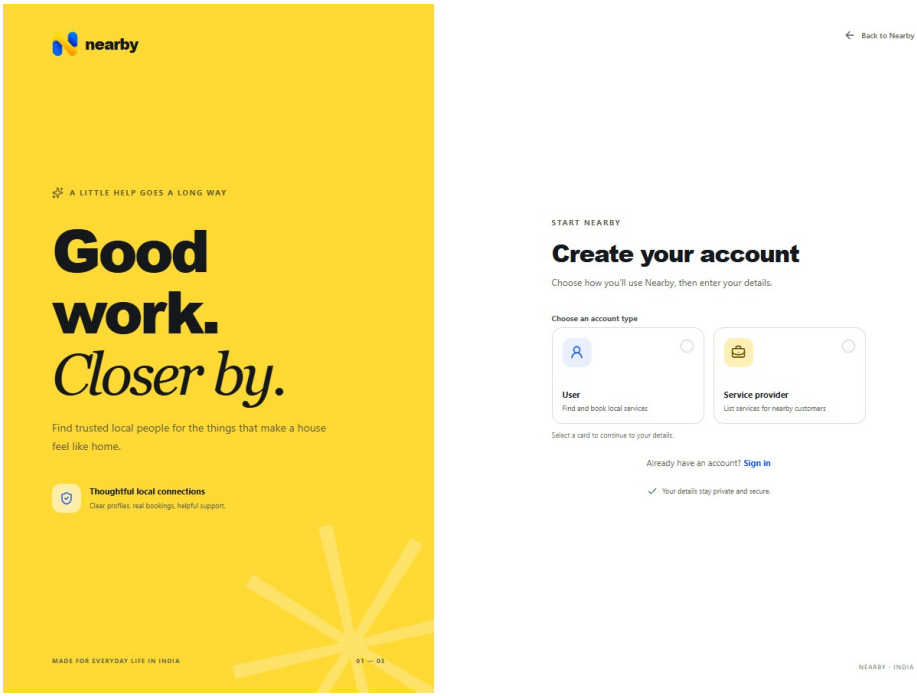


Figure 5.3 Registration role selection

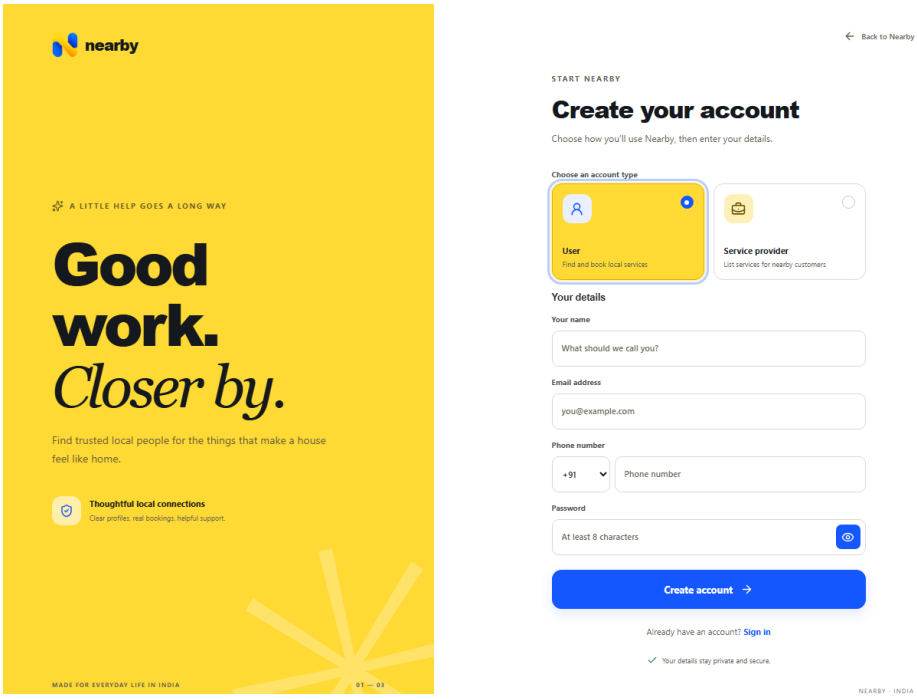


Figure 5.4 Customer registration form

Choosing the customer role reveals account entry fields. The screenshot records this UI interaction only and does not demonstrate persistence or a successful registration response.

Role selection reveals the account form. The server validates submitted identity and contact values and restricts public registration to customer and provider accounts.

Recovery and service discovery interfaces

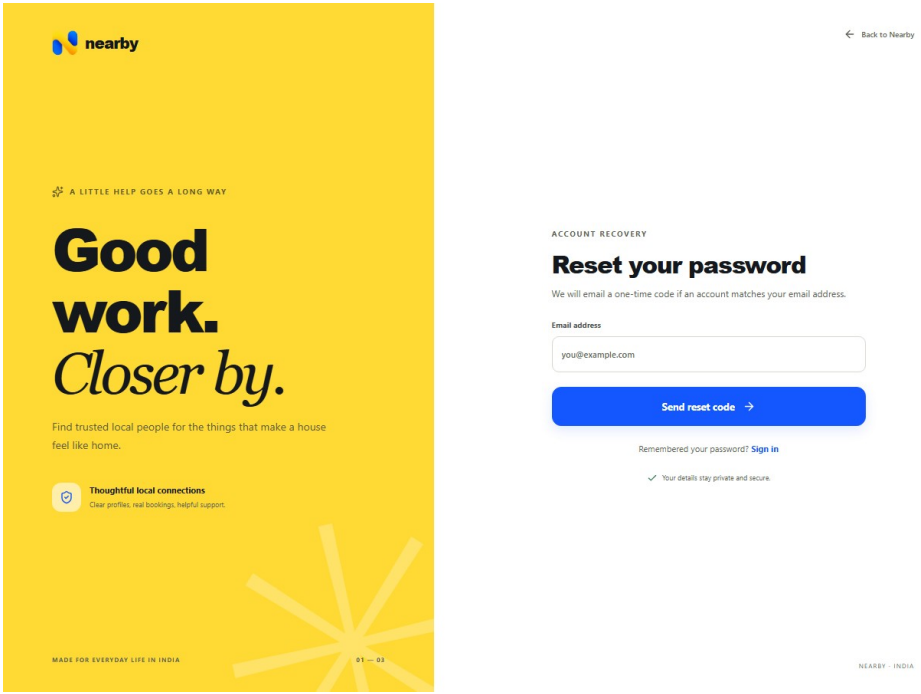


Figure 5.5 Password recovery page

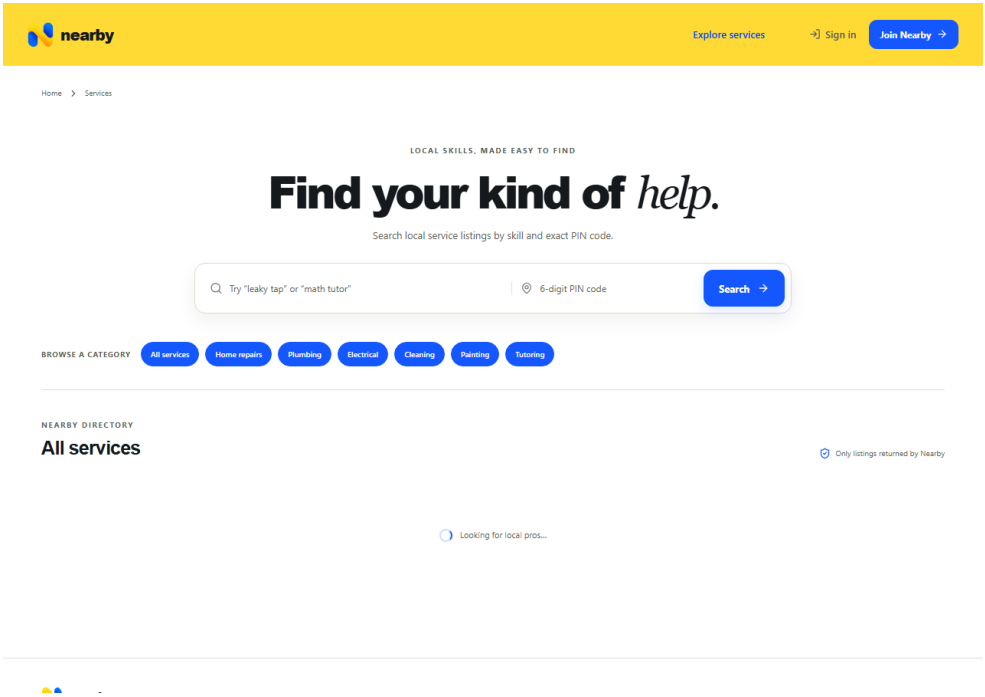


Figure 5.6 Service discovery page

The discovery capture includes the waiting state while the API is unavailable. Actual listing and booking results require a connected backend and suitable records.

Recovery depends on email delivery, while search depends on eligible service records. A rendered page does not establish that either backend workflow completed.

Pincode validation and protected output requirements

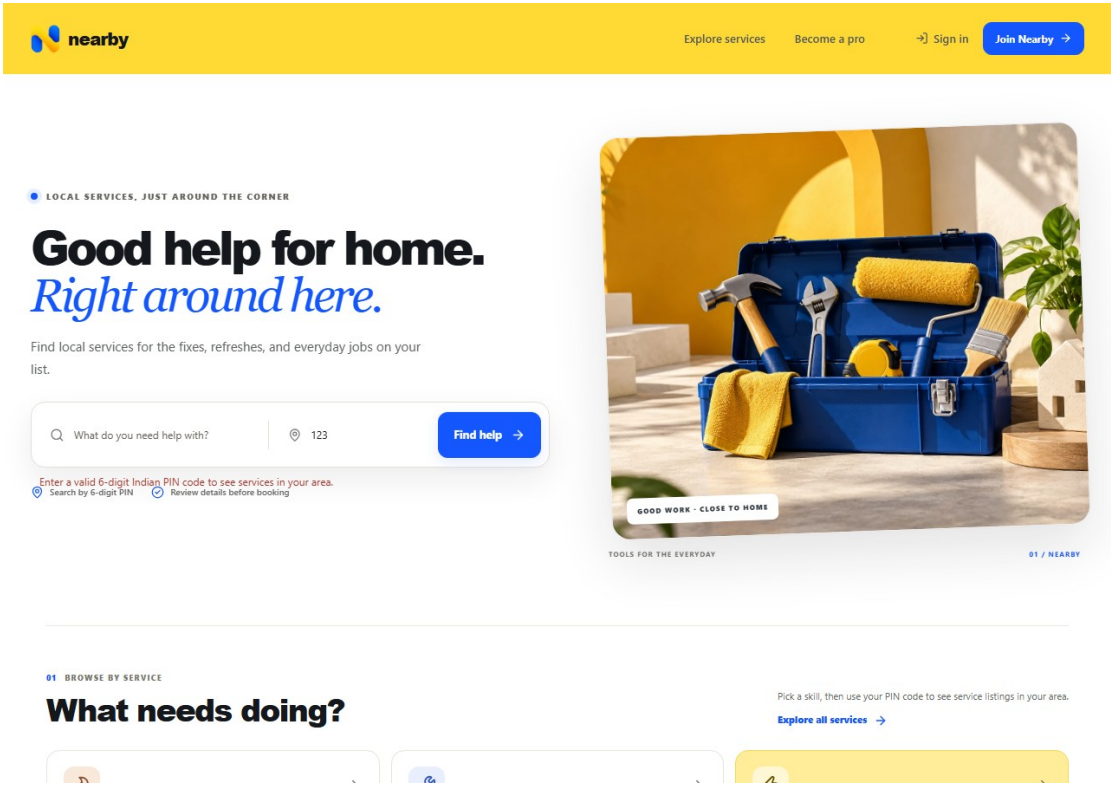


Figure 5.7 Pincode validation output

Submitting a three digit pincode displays the interface validation message. This confirms visible rejection of that input. It does not establish every search validation case or successful database discovery.

Table 5.1 Protected screenshot requirements

Screen	Required evidence
Customer and provider dashboards	Authenticated accounts and owned booking records
Administrator operations	Trusted admin account and appropriate records
Checkout and payout	Accepted booking and configured Razorpay sandbox
Booking chat and disputes	Two authorized participants and eligible booking state

5.2 TESTING AND VALIDATION

The existing backend suite was run with npm test on revision bc1ff9c on 22 October 2026. It reported 16 tests: 15 passed and one failed. The following table records the actual cases rather than converting proposed scenarios into claimed results. No coverage percentage or load benchmark was measured.

Table 5.2 Test cases and observed outcomes

ID	Existing test focus	Observed result
TC1	Model references and private selected fields	Passed
TC2	Unique six digit pincode validation	Passed
TC3	Rupees converted to integer paise	Passed
TC4	Demo categories and India calendar dates	Passed
TC5	Development KYC stays pending without documents	Passed
TC6	Authentication and role middleware responses	Passed
TC7	Admin registration and coordinate discovery	Failed at first assertion
TC8	Retained cancellation commission split	Passed
TC9	Full refund leaves no provider amount	Passed
TC10	Account safe reset and stored code digest	Passed
TC11	Expired reset code cannot update password	Passed
TC12	Five incorrect code attempts exhaust budget	Passed
TC13	One time reset hashing and token revocation	Passed
TC14	Password character and UTF 8 limits	Passed
TC15	Token version compatibility and reset change	Passed
TC16	Reset API rejects non six digit code	Passed

Interpretation of the failed assertion

The registration fixture omits the required phone field. The route validates phone before checking the public role allowlist, so the request receives HTTP 400 instead of the HTTP 403 expected by the test. This is a rejected request, but the fixture does not isolate the role restriction. Its later coordinate discovery assertion is not executed in this run because the first assertion fails.

Frontend checks

npm run lint reported one error and zero warnings: react-hooks/set-state-in-effect at Dashboard.jsx line 568, where setLoading(true) occurs synchronously inside an effect. npm run build passed, transforming 1961 modules and producing the browser bundle. A successful build does not override a failed lint check.

Additional scenarios requiring execution

An integrated environment should verify successful customer and provider registration, login, unauthorized cross account access, concurrent slot requests, checkout expiry, webhook signature failures, duplicated payment events, refund reconciliation, participant socket isolation and administrator dispute decisions. Email delivery and reset completion should be checked through an actual configured test mailbox. These scenarios are recommendations and are not recorded as passed tests.

Table 5.3 Test coverage by module

Module	Evidence present	Remaining verification
Accounts and recovery	Middleware and focused reset tests	Full browser login reset and SMTP delivery
Service discovery	Pincode and price unit checks	Database filtering and live listing results
Bookings and availability	Source inspection	Concurrent reservation hold and completion tests
Payments	Cancellation arithmetic tests	Sandbox orders webhooks refunds and transfers
Messages and disputes	Participant checks inspected in source	Two session isolation and admin outcomes
Frontend	7 public state captures and build	Mobile keyboard and protected route interaction

The matrix reports the type of evidence available, not a count of production features. A mocked API assertion cannot verify a real email, a database concurrency condition or an external gateway decision. Likewise, an interface image establishes visible rendering but not the correctness of every operation behind it.

Reproducibility

The source revision, commands and observed failure details are retained in this report so a later run can distinguish a changed implementation from a changed test fixture. The report generation did not edit application code. Future verification should use an isolated test database and preserve the same evidence separation when updating these results.

5.3 RESULTS AND DISCUSSION

Table 5.4 Objectives and outcomes

Objective	Current evidence	Limit
Structured discovery	Listings and pincode logic implemented	Live search not exercised
Controlled access	Middleware tests and route inspection	Cross user browser tests pending
Account recovery	Focused reset tests pass	SMTP and browser journey pending
Booking coordination	State models and routes implemented	Database concurrency tests pending
Financial handling	Arithmetic tests pass	External settlement not verified
User interface	Public captures and build pass	Lint issue and protected evidence remain

The project demonstrates a modular application with a coherent booking domain and visible public interfaces. The current evidence supports development progress while identifying a stale test fixture and one lint issue. It does not establish production readiness, complete end to end correctness, external service approval, measured response times or marketplace adoption.

Validation follow up

Update the registration test fixture so it reaches the role restriction, then exercise its coordinate search check. Resolve the dashboard lint finding and rerun the focused checks. Use an isolated development database for customer, provider and administrator journeys. Follow with Razorpay sandbox events, refunds, failed delivery behaviour, participant room isolation, concurrent booking requests and lifecycle recovery after restart. Record outcomes before making deployment claims.

Acceptance criteria for integrated testing

A complete customer journey should produce an owned booking for the selected service, pincode and time, then show the provider decision before checkout becomes available. A second account should be unable to read or change that booking unless it is the linked provider participant or has an explicitly authorized administrative operation. A successful test needs observable API and database results as well as the expected browser state.

Sandbox verification should capture event identifiers, payment state and booking transitions. Duplicate events should not create duplicate effects. Pending refunds must remain distinguishable from completed refunds, and payout must respect disputes. These are future acceptance criteria, not additional passing results.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

Nearby brings service discovery, provider availability, booking decisions and related communication into a centralized web application. The project uses explicit customer, provider and administrator roles and exact pincode coverage. Its data model links the service journey to booking records, allowing payment state, reviews and disputes to be understood in context.

The implementation applies full stack development concepts through React components, an Express API, MongoDB models, JWT access control and Socket.IO messaging. Password recovery adds expiring email codes and invalidation of earlier tokens. Integer money representation and recorded lifecycle states provide a foundation for more predictable marketplace operations.

Verification identified useful implemented behaviour together with clear limits. Fifteen backend tests passed, one backend assertion failed, lint reported one dashboard error and the web production build succeeded. Public pages were captured from the running app. Production KYC, external payment execution, email delivery and authenticated multi user workflows remain outside the verified results. These distinctions should be retained when presenting the project for academic review.

6.2 FUTURE ENHANCEMENTS

Future work should first close the existing verification gaps and configure the external integrations. New product capabilities should follow measured needs. The current evidence supports a development foundation and does not establish a live marketplace ready to receive payments.

Table 6.1 Roadmap of future work

Priority	Enhancement	Expected evidence
Immediate	Repair stale registration fixture and lint finding	Passing focused checks and reachable intended assertions
Immediate	Capture authenticated role journeys	Customer provider and admin browser results
Before launch	Select and implement production KYC vendor	Sandbox adapter integration with documented outcomes
Before launch	Exercise external payment lifecycle	Orders signed events refunds and held transfer evidence
Before launch	Configure SMTP and operations	Delivery retries monitoring backup and restore checks

Priority	Enhancement	Expected evidence
Next phase	Improve accessibility and mobile usability	Keyboard screen reader and viewport results
Optional	Evaluate distance based discovery	Separate decision and design beyond exact pincode
Optional	Develop analytics or native clients	Measured requirements and scoped validation

The marketplace requires external vendor configuration and operational support before live use. Current evidence includes no load benchmark, production deployment health check, disaster recovery exercise or usability study. Service coverage relies on configured pincodes, and no distance aware discovery is implemented. Protected workflow screenshots and browser evidence remain to be completed with a configured test environment.

Maps and distance matching would change the current pincode boundary and require their own product and data decisions. A native client should reuse stable, verified API contracts. Analytics should start from reliable operational records rather than projections of user growth or provider earnings.

Prioritized validation sequence

The first stage is to establish a reliable isolated development environment. Restore a clean test fixture for the required registration fields, resolve the dashboard lint finding and verify database readiness. Then exercise customer, provider and administrator accounts separately, retaining the relevant request and response evidence. A green build is useful at this stage because it proves bundle generation, but it should be reported alongside the test and lint outcomes.

The next stage is to validate operations across module boundaries. Scheduling tests should include two requests for an overlapping provider window, an unpaid hold that expires and a paid booking that reaches completion review. Communication tests should involve two authorized sessions and an unrelated third account. Verification tests should demonstrate that a development or disabled KYC reference cannot be approved as a live vendor result.

Only after those checks should an operator move to external sandbox processing and deployment recovery exercises. Record successful and failed webhook handling, email delivery and refund reconciliation. Review logging and backup recovery with the deployed configuration. Additional product features can then be prioritized using observed gaps and user feedback rather than assumed adoption or performance improvements.

REFERENCES AND BIBLIOGRAPHY

[1] React documentation. https://react.dev/

[2] Vite guide. https://vite.dev/guide/

[3] Node.js API documentation. https://nodejs.org/docs/latest/api/

[4] Express documentation. https://expressjs.com/

[5] MongoDB documentation. https://www.mongodb.com/docs/

[6] Mongoose documentation. https://mongoosejs.com/docs/

[7] Tailwind CSS with Vite. https://tailwindcss.com/docs/installation/using-vite

[8] React Router routing. https://reactrouter.com/start/declarative/routing

[9] Socket.IO 4 documentation. https://socket.io/docs/v4/

[10] RFC 7519 JSON Web Token. https://www.rfc-editor.org/rfc/rfc7519

[11] bcrypt.js project documentation. https://github.com/dcodeIO/bcrypt.js

[12] Zod documentation. https://zod.dev/

[13] Nodemailer documentation. https://nodemailer.com/

[14] Helmet documentation. https://helmet.js.org/

[15] Express Rate Limit overview. https://express-rate-limit.mintlify.app/overview

[16] Razorpay documentation. https://razorpay.com/docs/

[17] Vercel documentation. https://vercel.com/docs

[18] Render Node Express deployment guide. https://render.com/docs/deploy-node-express-app

Official URLs checked 22 October 2026. Project evidence: local Nearby source and tests at revision bc1ff9c. Technology documentation does not establish application runtime behaviour.

Project evidence sources

The implementation descriptions draw on the local README, server domain modules, middleware, Mongoose models and frontend route and service files. The automated results come from the existing server tests, frontend lint command and Vite production build run on 22 October 2026. The screenshots were captured from the local browser application; they establish visible public interface states and do not represent authenticated bookings or live payments.

The official references above explain the selected tools and protocols. They should not be cited as proof that Nearby meets a security, availability or performance target. For reproducibility, later report revisions should identify the checked source revision, executed commands, configured test environment and external sandbox outcomes. Private environment values, access tokens, passwords and vendor documents should remain outside the report and its source excerpts.

APPENDICES

Source Code

1 Project structure

Nearby separates the server and browser application. The following selected tree identifies the main implementation areas. Environment files and the local demo access file contain private configuration and are excluded from the report.

Selected repository structure

```
server/
  package.json
  src/
    app.js                HTTP middleware and route groups
    index.js              Database HTTP Socket.IO and lifecycle startup
    config/db.js          MongoDB connection
    middleware/auth.js    Identity token version and roles
    modules/
      auth/               Registration login and password recovery
      users/              User model and account profile
      providers/          Provider profiles and KYC adapter
      services/           Listings search and integer price helpers
      availability/       Windows and provider day reservations
      bookings/           Request acceptance checkout completion
      payments/           Payment events refunds and transfers
      messaging/          Booking REST and Socket.IO messages
      reviews/            Customer completed booking reviews
      disputes/           Participant complaints and resolution
      admin/              Audit and marketplace operations
      settings/           Marketplace policy and readiness
    utils/                Lifecycle validation and response helpers
  tests/                  Existing focused Node tests
  scripts/                Admin provisioning and guarded demo seed
website/
  package.json
  src/
    App.jsx               Public and protected routes
    pages/                Landing search detail auth and dashboards
    components/           Shared application layout and states
    services/             Browser API and session context
    assets/               Project image assets
    index.css             Brand and layout foundations
    App.css               Component and page styles
```

2 Package installation

```
# Terminal 1 from the project root
cd server
npm install
Copy-Item .env.example .env
# Configure private MongoDB and JWT values locally
npm run dev

# Terminal 2 from the project root
cd website
npm install
Copy-Item .env.example .env
# VITE_API_URL=http://localhost:4000/api/v1
npm run dev
```

The API readiness URL is http://localhost:4000/health and the web development URL is http://localhost:5173. Readiness depends on a connected database and successful ping. The guarded demo seed requires a fresh disposable database; it was not run for this report.

3 MongoDB connection

server/src/config/db.js lines 1 to 28

```
const mongoose = require("mongoose");
const dns = require("node:dns");
const net = require("node:net");

function configureDnsResolvers() {
  const configured = process.env.MONGODB_DNS_SERVERS;
  if (!configured) return;

  const servers = configured.split(",").map((server) => server.trim());
  if (!servers.length || servers.some((server) => !net.isIP(server))) {
      throw new Error("MONGODB_DNS_SERVERS must contain comma-separated IP
addresses.");
  }

  dns.setServers(servers);
}

async function connectdb(dbUrl) {
  if (!dbUrl) {
    throw new Error("MONGO_URI is required to start the API.");
  }

  configureDnsResolvers();
  await mongoose.connect(dbUrl, {
    serverSelectionTimeoutMS: 10000,
    connectTimeoutMS: 10000,
  });

  return mongoose.connection;
```

4 Express server and realtime engine

server/src/index.js lines 1 to 23

```
require("dotenv").config();

const http = require("node:http");
const mongoose = require("mongoose");
const { Server } = require("socket.io");
const connectdb = require("./config/db");
const { app, allowedOrigins } = require("./app");
const { attachMessageSocket } = require("./modules/messaging/socket");
const { startLifecycleWorker } = require("./utils/lifecycle");

const port = Number(process.env.PORT || 4000);

async function startServer() {
  if (!Number.isInteger(port) || port < 1 || port > 65535) {
    throw new Error("PORT must be an integer between 1 and 65535.");
  }
```

```
  // MONGO_URI is canonical; keep the old key as a local migration fallback.
  await connectdb(process.env.MONGO_URI || process.env.MONGODB_URI);

  const server = http.createServer(app);
  const io = new Server(server, {
    cors: {
```

The remaining startup logic coordinates listening, background lifecycle work and shutdown. The mounted application routes and Socket.IO share the server environment. A frontend development start does not prove that this database dependent process is ready.

5 Environment configuration

Illustrative variable names only

```
PORT=4000
MONGO_URI=<configure privately>
JWT_SECRET=<configure a strong private secret>
CLIENT_ORIGINS=http://localhost:5173
# SMTP and Razorpay secrets belong only on the server.
# Browser configuration contains VITE_API_URL only.
```

6 User model

server/src/modules/users/user.model.js lines 10 to 23

```
passwordResetCodeHash: { type: String, select: false, default: undefined },
  passwordResetExpiresAt: { type: Date, select: false, default: undefined },
  passwordResetAttempts: { type: Number, select: false, default: 0, min: 0, max: 5 },
  role: { type: String, enum: ["customer", "provider", "admin"], required: true, default:
"customer" },
  isActive: { type: Boolean, default: true, required: true },
}, { timestamps: true, versionKey: false });

userSchema.index({ email: 1 }, { unique: true });

module.exports = mongoose.models.User || mongoose.model("User", userSchema);
```

7 Authentication controller and password recovery

server/src/modules/auth/password-reset.js lines 6 to 25

```
const PASSWORD_RESET_TTL_MS = 10 * 60 * 1000;
const PASSWORD_RESET_MAX_ATTEMPTS = 5;
const PASSWORD_RESET_REQUEST_MESSAGE = "If an account exists for this email,
a reset code will be sent.";
const PASSWORD_RESET_FIELDS = {
  passwordResetCodeHash: 1,
  passwordResetExpiresAt: 1,
  passwordResetAttempts: 1,
};

function hasPasswordResetSecret(secret = process.env.JWT_SECRET) {
  return typeof secret === "string" && secret.length >= 32;
}

function hashPasswordResetCode(code, secret = process.env.JWT_SECRET) {
  if (!hasPasswordResetSecret(secret)) return null;
```

```
  return crypto.createHmac("sha256", secret).update(String(code)).digest("hex");
}

function createPasswordResetCode() {
  return crypto.randomInt(0, 1_000_000).toString().padStart(6, "0");
```

Reset state is stored with a digest rather than a plaintext code. Confirmation consumes the active reset and increments the token version. These excerpts must be read with the full module; they are not an alternative implementation.

8 Role authorization middleware

server/src/middleware/auth.js selected lines 38 to 60

```
}
    if ((claims.ver ?? 0) !== tokenVersion) {
        return sendError(res, 401, "INVALID_TOKEN", "The access token is invalid or expired.");
    }

    req.user = { id: String(user._id), role: user.role, email: user.email };
    return next();
  } catch (error) {
    console.error("Authentication lookup failed:", error.message);
      return sendError(res, 500, "AUTH_LOOKUP_FAILED", "Unable to validate the account.");
  }
// Intervening source lines omitted
function requireRole(roleOrRoles) {
  const roles = Array.isArray(roleOrRoles) ? roleOrRoles : [roleOrRoles];
  return (req, res, next) => {
      if (!req.user) return sendError(res, 401, "UNAUTHENTICATED", "Sign in to continue.");
    if (!roles.includes(req.user.role)) return sendError(res, 403, "FORBIDDEN", "You do not have access to this action.");
    return next();
  };
}

module.exports = { requireAuth, requireRole };
```

9 Service discovery and listing rules

Public search is based on exact pincode coverage. Service ownership and provider role protect modifications. Prices are represented as integer paise and a listing includes a duration used in booking validation. No map or radius fallback is implemented.

server/src/modules/services/pricing.js lines 1 to 13

```
function priceToPaise(price) {
  if (typeof price === "number") {
    if (!Number.isFinite(price) || price <= 0) return null;
    price = String(price);
  }
  if (typeof price !== "string") return null;

  const match = /^(0|[1-9]\d*)(?:\.(\d{1,2}))?$/.exec(price.trim());
  if (!match) return null;

  const rupees = BigInt(match[1]);
  const paiseDigits = (match[2] || "").padEnd(2, "0");
```

```
  const paise = rupees * 100n + BigInt(paiseDigits || "0");
```

10 Availability and scheduling

server/src/modules/availability/service.js selected lines 8 to 65

```
return new Date(`${date}T${time}:00${INDIA_OFFSET}`);
}

function isDateKey(value) {
  if (typeof value !== "string" || !/^\d{4}-\d{2}-\d{2}$/.test(value)) return false;
  const parsed = new Date(`${value}T00:00:00.000Z`);
   return Number.isFinite(parsed.getTime()) && parsed.toISOString().slice(0, 10) ===
value;
}

async function getProviderWindow(providerId, startsAt, endsAt) {
// Intervening source lines omitted
  const now = new Date();
  const noOtherOverlap = {
    provider: providerId,
    dateKey: local.dateKey,
    reservations: {
      $not: {
        $elemMatch: {
          booking: { $ne: bookingId },
          startsAt: { $lt: endsAt },
          endsAt: { $gt: startsAt },
          $or: [
            { state: { $in: ["confirmed", "completed"] } },
            { state: "held", expiresAt: { $gt: now } },
          ],
        },
      },
    },
```

11 Booking lifecycle records

server/src/modules/bookings/booking.model.js lines 14 to 27

```
enum: ["requested", "accepted", "checkout_expired", "awaiting_payment", "confirmed",
"completion_pending", "completed", "rejected", "cancelled", "disputed"],
      default: "requested",
      index: true,
    },
    pricePaise: { type: Number, required: true, min: 1 },
    durationMinutes: { type: Number, required: true, min: 1 },
    slotHoldExpiresAt: { type: Date, default: null, index: true },
    completionMarkedAt: { type: Date, default: null },
    completionMarkedBy: { type: mongoose.Schema.Types.ObjectId, ref: "User", default:
null },
    reviewDeadlineAt: { type: Date, default: null, index: true },
    completedAt: { type: Date, default: null },
    cancelledAt: { type: Date, default: null },
    cancellationReason: { type: String, trim: true, maxlength: 1000, default: "" },
  },
```

12 Payment and refund operations

Payment processing combines booking state, external identifiers, signed events and refund reconciliation. The following worker excerpt shows the scheduled coordination points. It does not establish that an external refund or transfer completed.

server/src/utils/lifecycle.js lines 112 to 128

```
async function startLifecycleWorker() {
  let running = false;
  const tick = async () => {
    if (running || mongoose.connection.readyState !== 1) return;
    running = true;
    try {
      await expireCheckoutHolds();
      await completeMatureJobs();
      await require("../modules/payments/service").reconcilePendingRefunds();
    } catch (error) {
      console.error("Booking lifecycle worker failed:", error.message);
    } finally {
      running = false;
    }
  };
  const timer = setInterval(() => { void tick(); }, 30_000);
  timer.unref?.();
```

13 Booking communication

server/src/modules/messaging/routes.js lines 19 to 28

```
async function createMessage(bookingId, senderId, rawText) {
  ensure(isObjectId(bookingId), 400, "Invalid booking ID", "INVALID_BOOKING_ID");
  const text = typeof rawText === "string" ? rawText.trim() : "";
  ensure(text.length > 0 && text.length <= 4000, 400, "Message must be between 1 and
4000 characters", "INVALID_MESSAGE");
  const booking = await Booking.findById(bookingId);
  ensure(booking, 404, "Booking not found", "BOOKING_NOT_FOUND");
  ensure(MESSAGE_STATUSES.includes(booking.status), 409, "Messaging opens after a
provider accepts the booking", "MESSAGING_NOT_AVAILABLE");
  ensure(await canParticipate(booking, { id: senderId }), 403, "Only booking participants
can access this conversation", "FORBIDDEN");
  const message = await Message.create({ booking: booking._id, sender: senderId, text
});
   const populated = await Message.findById(message._id).populate("sender", "name
role").lean();
```

14 Reviews and dispute handling

A customer review requires a completed owned booking. Disputes remain tied to the customer and provider context, and administrative resolution affects payout eligibility. These operations need a connected database and authorized records for end to end verification.

server/src/modules/reviews/routes.js lines 9 to 18

```
router.post(
  "/",
  requireAuth,
  requireRole("customer"),
  asyncHandler(async (req, res) => {
    const { bookingId, rating, comment = "" } = req.body || {};
       ensure(isObjectId(bookingId), 400, "bookingId must be a valid booking ID",
"INVALID_BOOKING_ID");
    ensure(Number.isInteger(rating) && rating >= 1 && rating <= 5, 400, "rating must
be an integer from 1 to 5", "INVALID_RATING");
    ensure(typeof comment === "string" && comment.length <= 2000, 400, "comment
cannot exceed 2000 characters", "INVALID_COMMENT");
    const booking = await Booking.findOne({ _id: bookingId, customer: req.user.id });
```

15 Frontend routing and session access

website/src/App.jsx lines 17 to 24

```
}

function ProtectedPage({ children, role }) {
  const { user, ready } = useSession()
  const location = useLocation()
  if (!ready) return <LoadingState full label="Getting Nearby ready..." />
  if (!user) return <Navigate to="/login" replace state={{ from: `${location.pathname}
${location.search}` }} />
  if (role && getRole(user) !== role) return <Navigate to={homeForRole(user)} replace
/>
```

16 REST API endpoints

Table 6.2 Representative endpoints

Method and route	Purpose
POST /api/v1/auth/login	Authenticate account
POST /api/v1/auth/register	Create public customer or provider account
POST /api/v1/auth/password-reset/request	Request reset code
POST /api/v1/auth/password-reset/confirm	Consume reset and update password
GET /api/v1/services	Search service listings
PUT /api/v1/availability/me	Publish provider windows
POST /api/v1/bookings	Request booking
POST /api/v1/bookings/:bookingId/accept	Accept the identified booking
POST /api/v1/payments/webhook	Receive signed payment event
GET /api/v1/messages/:bookingId	Read participant conversation
POST /api/v1/disputes	Raise an eligible complaint
GET /health	Database readiness check

17 Testing and verification

Run npm test in server, then npm run lint and npm run build in website. The recorded run returned 15 passing tests and one failure, one lint error and a successful build. The failure and coverage boundaries are explained in Chapter 5. Live payment, KYC, SMTP and authenticated multi user tests were not completed and must not be reported as passed.